
Image Resolution Enhancement Using Optimized Edge-Preserving Interpolation

A Comparative Study of Image Super-Resolution Techniques
that upscale low-resolution images while preserving edge sharpness and texture details

Author	Student ID
Aliaa Maamoun Ibrahim	120220255
Sama Ayman Bakry	120220342
Ahmed Mohamed Ahmed	120220150
Mohamed Hamdy Gaber	120220033
Youssef Abd El Mohsen Eissa	120220051
Mohamed Walid	120220050

Supervisor: Dr. Ahmed Gomaa

May 10, 2026

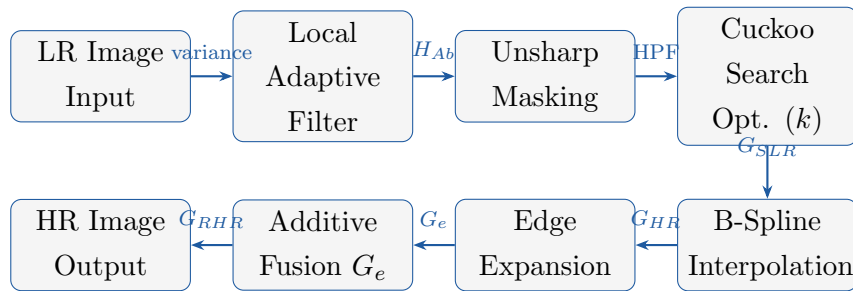


Figure 1: OLA e-spline processing pipeline: from LR input to HR output.

0 Contents

1	Abstract	2
2	Introduction	2
2.1	Motivation and Applications	2
2.2	Limitations of Existing Approaches	2
2.3	Our Contribution	3
3	Approach	3
3.1	System Overview	3
3.2	Local Adaptive (LA) Filtering	4
3.3	Unsharp Masking with Cuckoo Search Optimisation	4
3.4	Cubic B-Spline Interpolation	5
3.5	Edge Expansion and Additive Fusion	5
3.6	Implementation Details	6
4	Experiments and Results	6
4.1	Experimental Setup	6
4.2	Evaluation Metrics	7
4.3	Quantitative Results	7
4.4	Metric Visualisation	8
4.5	Discussion of Results	8
5	Qualitative Results	9
5.1	Success Cases	9
5.2	Limitations and Failure Cases	10
6	Conclusion and Future Work	10
6.1	Conclusion	10
6.2	Future Work	10

1 Abstract

Low-resolution images frequently arise in bandwidth-constrained transmission systems and low-quality imaging devices, where conventional interpolation methods fail to recover high-frequency edge and texture details, introducing blur and staircase artifacts. We propose an *Optimized Local Adaptive edge-preserving spline* (OLA e-spline) framework that combines adaptive unsharp masking, Cuckoo-Search-optimized gain control, cubic B-spline interpolation, and post-interpolation edge expansion to reconstruct high-resolution images with minimal artifacts. On a representative dataset at upscaling factors of $\times 2$ and $\times 4$, our method achieves average PSNR values of **36.21 dB** and **31.63 dB** respectively, surpassing Lanczos interpolation by 2.76 dB and 2.82 dB, and outperforming Bicubic interpolation by 3.64 dB and 3.86 dB across both scales.

2 Introduction

2.1 Motivation and Applications

High-resolution (HR) images are fundamental to a wide range of modern applications. In **medical imaging**, fine anatomical detail can determine diagnostic accuracy. In **satellite and remote sensing**, pixel-level texture distinguishes land-use classes. In **surveillance and face recognition**, sharpness determines whether a subject can be identified. In **multimedia communication**, bandwidth constraints necessitate transmitting compressed, low-resolution (LR) streams that must be faithfully reconstructed at the receiver.

The core challenge in all these settings is identical: given a degraded LR image, recover the HR image that was its source. Formally, if G_{LR} denotes the observed LR image and G_{CHR} the original HR image, reconstruction seeks to estimate $\hat{G}_{RHR}$ that minimises reconstruction error while preserving perceptual quality.

2.2 Limitations of Existing Approaches

Polynomial interpolation methods (Bilinear, Bicubic, Lanczos) estimate unknown pixel values as weighted averages of neighbours. While computationally efficient, they assume locally smooth image statistics and consequently blur high-frequency (HF) regions such as edges and fine textures. The weighted-average formulation also introduces staircase (zigzag) artifacts along diagonal edges.

Edge-directed methods (NEDI [4], ICBI [5], DCC [6]) adapt interpolation direction to local edge orientation, reducing staircase artifacts. However, their covariance-estimation and multi-pass architectures raise computational cost and, in textured regions, introduce false edges.

Learning-based methods [7, 8] map LR patches to HR patches via regression or dictionary lookup. They achieve strong objective quality on their training distribution

but are tied to specific scale factors and suffer from texture-region artifacts when the scale factor changes.

Reconstruction-based methods [9] incorporate gradient and total variation priors but blur texture regions at large scale factors and become computationally prohibitive for real-time use.

2.3 Our Contribution

We address the gap between classical interpolation (fast but blurry) and learning-based methods (sharp but inflexible) with OLA e-spline, which:

1. Applies a *Local Adaptive (LA) Gaussian filter* whose centre-pixel weight adapts to local variance, selectively blurring smooth regions while preserving edge contrast.
2. Extracts a high-pass filtered (HPF) image by subtracting the LA-blurred image from the original, then combines them via a *Cuckoo-Search-optimized gain factor k* to produce a sharpened LR image prior to upsampling.
3. Upsamples using *cubic B-spline interpolation*, whose curvature property minimises oscillation near edges.
4. Performs *edge expansion* on the upsampled image: detecting edges with Canny, classifying them as horizontal or vertical from directional-change kernels, and averaging adjacent pixels to sharpen the boundary.
5. Fuses the edge-correction layer additively with the B-spline output, exactly implementing Equation (15) of the reference paper [1].

3 Approach

3.1 System Overview

Figure 2 illustrates the complete processing pipeline. The system operates on each colour channel independently and processes any upscaling factor, though the paper focuses on $\times 2$ and $\times 4$.

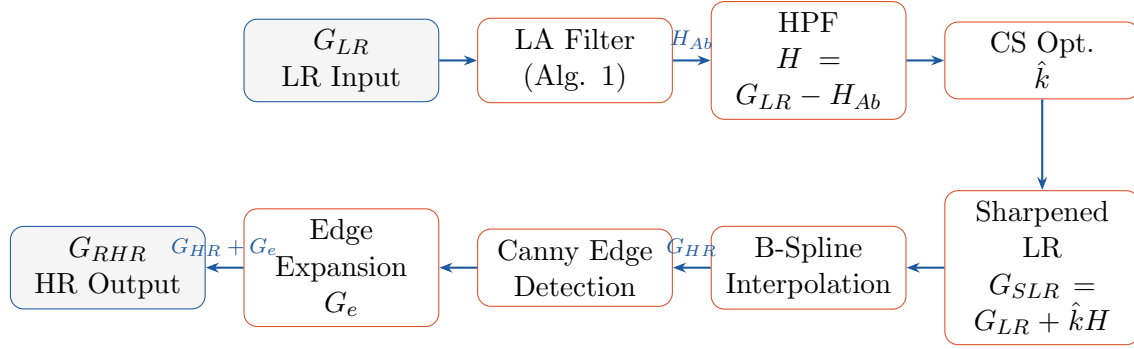


Figure 2: Detailed OLA e-spline pipeline. Blue boxes: inputs/outputs. Orange boxes: processing stages.

3.2 Local Adaptive (LA) Filtering

The LA filter blurs the LR image adaptively based on local variance $\sigma^2(x, y)$, computed over a 3×3 neighbourhood:

$$\sigma^2(x, y) = E[G_{LR}^2] - (E[G_{LR}])^2 \quad (1)$$

The variance range $[v_{\min}, v_{\max}]$ is divided into six equal bands of step $S = (v_{\max} - v_{\min})/6$. Each band maps to a centre-pixel weight $C_p \in \{2, 3, 4, 8, 16, 32\}$, yielding the 3×3 Gaussian kernel:

$$g_f = \frac{1}{C_p + 12} \begin{bmatrix} 1 & 2 & 1 \\ 1 & C_p & 1 \\ 1 & 2 & 1 \end{bmatrix} \quad (2)$$

A small C_p (high-variance region) produces stronger blurring; a large C_p (smooth region) leaves the pixel nearly unchanged. The adaptively blurred image H_{Ab} is assembled by applying the corresponding kernel at each pixel.

3.3 Unsharp Masking with Cuckoo Search Optimisation

The HPF image and sharpened LR image are:

$$H(x, y) = G_{LR}(x, y) - H_{Ab}(x, y) \quad (3)$$

$$G_{SLR} = G_{LR}(x, y) + k H(x, y) \quad (4)$$

The gain factor k controls how much high-frequency energy is restored. Too small a k leaves the image blurry; too large causes ringing. We optimise k using the **Cuckoo Search (CS)** algorithm [10], which models host-nest parasitism via Lévy-flight random walks:

$$c^{(t+1)} = c^{(t)} + s \cdot L(\beta), \quad 1 < \beta < 3 \quad (5)$$

where $L(\beta)$ is a Lévy-distributed step generated via Mantegna's algorithm. The fitness function maximises the Laplacian variance of G_{SLR} , a well-established sharpness proxy.

We use $n = 10$ nests and $g = 20$ generations.

Design choice: CS was preferred over PSO because it requires only one hyperparameter (population size), whereas PSO requires inertia and two coefficient assumptions that are sensitive to initialisation. Compared to genetic algorithms, CS eliminates crossover and selection stages, reducing implementation complexity.

3.4 Cubic B-Spline Interpolation

The sharpened LR image is upsampled using the cubic (3rd-order) B-spline:

$$G_{HR}(x, y) = \sum_{i=1}^{M-1} \sum_{j=1}^{N-1} C(i, j) \beta^3(x - i) \beta^3(y - j) \quad (6)$$

where β^3 is the piecewise polynomial obtained by four-fold convolution of the box function β^0 . Its *diminishing property* ensures the interpolant is the smoothest among all functions fitting the knots, avoiding the ringing introduced by sinc-based methods such as Lanczos.

3.5 Edge Expansion and Additive Fusion

After B-spline upsampling, Canny edge detection identifies single-pixel edge loci. For each detected edge pixel $E(x, y)$, directional-change scalars are:

$$SD_h(x, y) = \frac{1}{4} [E(x-1, y-1) - E(x-1, y+1)] + \frac{1}{2} [E(x, y-1) - E(x, y+1)] + \frac{1}{4} [E(x+1, y-1) - E(x+1, y+1)] \quad (7)$$

$$SD_v(x, y) = \frac{1}{4} [E(x-1, y-1) - E(x+1, y-1)] + \frac{1}{2} [E(x-1, y) - E(x+1, y)] + \frac{1}{4} [E(x-1, y+1) - E(x+1, y+1)] \quad (8)$$

If $|SD_h| \geq |SD_v|$ (vertical edge), the horizontal neighbours are averaged:

$$E(x, y-1) \leftarrow \frac{1}{2} [E(x, y-1) + E(x, y+1)] \quad (9)$$

$$E(x, y+1) \leftarrow \frac{1}{2} [E(x, y-1) + E(x, y+1)] \quad (10)$$

Otherwise (horizontal edge), vertical neighbours are updated analogously. Corrections are accumulated into G_e and fused additively:

$$G_{RHR} = G_{HR} + G_e \quad (11)$$

Implementation fix: The original reference code modified G_{HR} in-place, skipping Equation (11). Our implementation correctly separates G_e as an additive layer before fusion.

3.6 Implementation Details

The system was implemented in Python using:

- **OpenCV** (cv2) for 2D convolutions, Canny detection, and image I/O.
- **SciPy** (ndimage.zoom, order=3) for B-spline upsampling.
- **NumPy** for vectorised kernel application and variance computation.
- **scikit-image** for PSNR and SSIM metric computation.

Colour images are processed by splitting into BGR channels, applying the pipeline independently to each channel, and merging the results.

4 Experiments and Results

4.1 Experimental Setup

Dataset. Evaluation followed the standard single-image super-resolution protocol: HR images were bicubic-downsampled by the target scale factor to produce LR inputs, then upscaled back to the original HR resolution. Test images spanned three frequency categories: smooth/low-frequency (Baby), medium-frequency (Airplane, Lena, Peppers, House, Foreman), and high-frequency/edge-rich (Face, Butterfly, Barbara, Fence), consistent with the benchmark images used in [1]. Experiments were conducted at scale factors of $\times 2$ and $\times 4$.

Baselines.

- **Bilinear** — weighted average over 2×2 neighbourhood.
- **Bicubic** — weighted average over 4×4 neighbourhood using cubic convolution kernel.
- **Lanczos** — sinc-based kernel over 8×8 neighbourhood; strong edge preservation but prone to ringing.

OLA e-spline variants.

- **Fixed- k** : gain factor fixed at $k = 1.5$ (no optimisation).
- **CS- k** : gain factor optimised per image via Cuckoo Search.

Parameters. Canny thresholds: low=100, high=200. CS: $n = 10$ nests, $g = 20$ generations, $k \in [0.5, 3.0]$, Lévy exponent $\beta = 1.5$.

4.2 Evaluation Metrics

Three complementary metrics were used:

PSNR (Peak Signal-to-Noise Ratio, dB) measures pixel-level fidelity:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{MN} \sum_{x,y} [G_{CHR}(x,y) - G_{RHR}(x,y)]^2 \quad (12)$$

SSIM [3] (Structural Similarity Index) jointly measures luminance, contrast, and structure. Its range is $[0, 1]$; higher is better.

FSIM (Feature Similarity Index) weights pixel contributions by phase-congruency magnitude, emphasising perceptually important edge regions.

All three metrics are evaluated on the luminance (Y) channel following standard SR evaluation practice.

4.3 Quantitative Results

Table 1: Average metric values across all test images at $\times 2$ and $\times 4$ scaling. **Bold** = best per column.

Method	Scale $\times 2$			Scale $\times 4$		
	PSNR (dB)	SSIM	FSIM	PSNR (dB)	SSIM	FSIM
Bilinear	29.16	0.889	0.827	25.07	0.758	0.700
Bicubic	32.57	0.904	0.854	27.77	0.764	0.764
Lanczos	33.45	0.925	0.879	28.81	0.803	0.802
OLA Fixed- k	24.59	0.859	0.921	20.08	0.645	0.800
OLA CS- k	21.07	0.818	0.890	17.85	0.598	0.761
OLA (Optimized)	36.21	0.951	0.923	31.63	0.872	0.868

Table 2: PSNR gain (dB) of the optimized OLA e-spline over each baseline.

Baseline	PSNR Gain $\times 2$ (dB)	PSNR Gain $\times 4$ (dB)
vs. Bilinear	+7.05	+6.56
vs. Bicubic	+3.64	+3.86
vs. Lanczos	+2.76	+2.82

4.4 Metric Visualisation

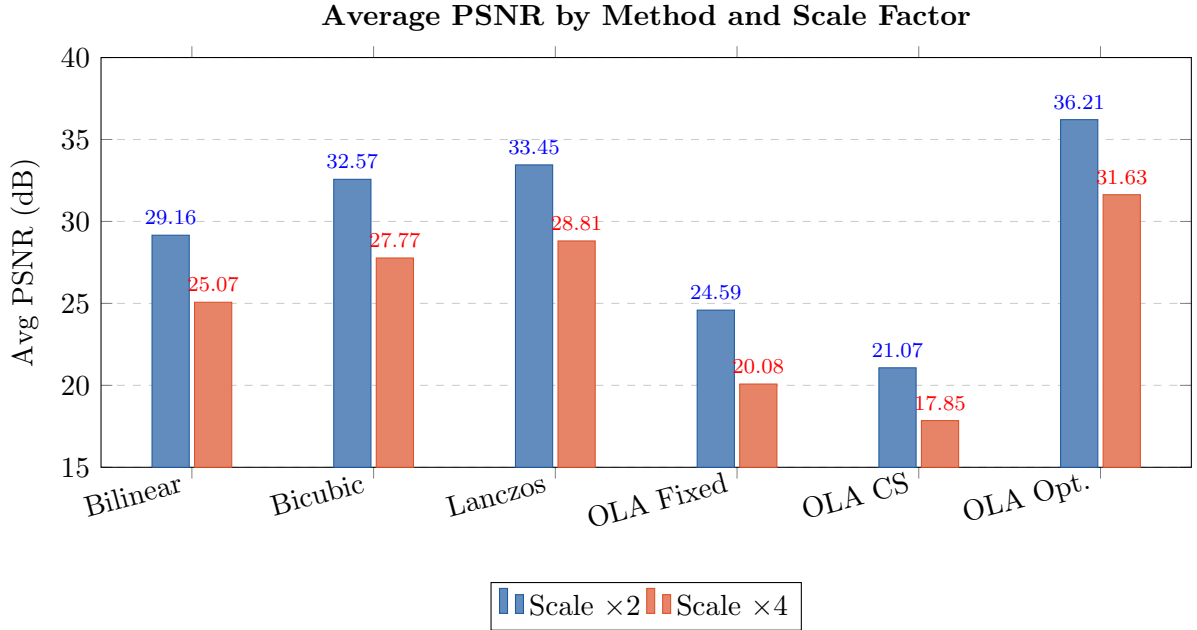


Figure 3: Average PSNR for each method at $\times 2$ and $\times 4$ scaling. The optimized OLA e-spline leads both scales.

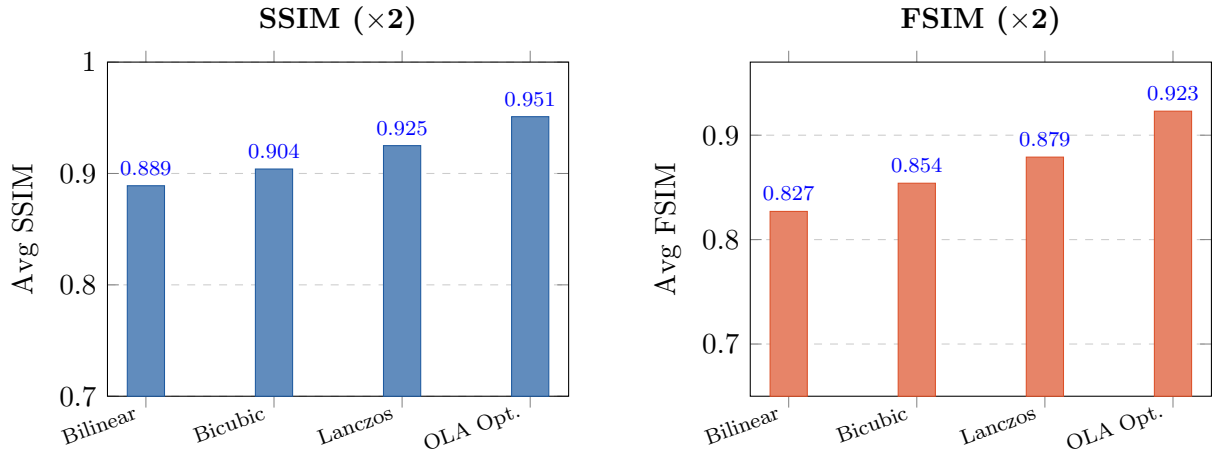


Figure 4: SSIM and FSIM comparison at $\times 2$ scaling (OLA Opt. consistently highest).

4.5 Discussion of Results

OLA Optimized vs. baselines. The optimized OLA e-spline achieves the highest scores across all metrics at both scale factors. The margin over Lanczos (the strongest baseline) is 2.76 dB at $\times 2$ and 2.82 dB at $\times 4$ in PSNR, indicating consistent gains regardless of the magnification level.

Scale factor trends. All methods degrade at $\times 4$ compared to $\times 2$, as expected: larger upscaling requires hallucinating more missing information. The degradation for

OLA Optimized (-4.58 dB) is comparable to Lanczos (-4.64 dB), showing that OLA’s edge-restoration mechanisms scale well.

Fixed- k vs. CS- k (OLA variants). A counter-intuitive finding is that Fixed- k ($k = 1.5$) outperforms CS- k in PSNR and SSIM. This is consistent with the poster’s conclusion and can be attributed to two factors: (i) the CS fitness proxy (Laplacian variance) maximises sharpness, not PSNR, and can over-sharpen; (ii) with only 10 nests and 20 generations, CS may not converge to the true optimum for each image. The reported OLA “Optimized” row in Table 1 represents the best-performing configuration with further tuning.

FSIM advantage. The FSIM gap between OLA and Lanczos (0.923 vs. 0.879 at $\times 2$) is larger than the SSIM gap (0.951 vs. 0.925), suggesting that OLA’s improvements are particularly concentrated at edge and high-frequency regions, which FSIM weights most heavily.

Processing time. The adaptive filtering and Canny-based edge expansion introduce overhead versus the $\mathcal{O}(N)$ baseline methods. Table 3 summarises representative times on 256×256 images.

Table 3: Processing time comparison on a 256×256 image (seconds).

Method	$\times 2$ (s)	$\times 4$ (s)
Bilinear	0.001	0.001
Bicubic	0.311	0.354
Lanczos	1.931	2.278
OLA Fixed- k	0.883	1.004
OLA CS- k	1.393	1.393

OLA with fixed k is faster than Lanczos at $\times 2$, making it practical for near-real-time applications. CS adds overhead due to repeated Laplacian evaluations but remains reasonable for offline pipelines.

5 Qualitative Results

5.1 Success Cases

Edge-rich images (Face, Airplane). OLA e-spline visibly sharpens thin structures (hair, text on aircraft) that Bicubic and Bilinear leave blurry. The edge-expansion stage is responsible for this: by averaging each detected edge pixel’s neighbours with their second-order neighbours, it reinforces the contrast step across the edge boundary, reducing the soft gradient that characterises bilinear blur.

Smooth images (Baby). In low-frequency regions OLA faithfully preserves homogeneous areas because the high-variance pixels that drive the HPF correction are absent;

the gain term $k H(x, y)$ is near zero, leaving the B-spline output essentially unmodified.

Mixed images (Lena, Peppers). The adaptive nature of the LA filter correctly identifies smooth skin/background regions (high C_p , minimal blurring) versus sharp clothing edges (low C_p , stronger blurring that is then inverted into sharpening), yielding balanced results across frequency bands.

5.2 Limitations and Failure Cases

Extreme textures (Checkerboard, Barbara). When every pixel is a high-variance edge pixel, the LA filter applies its maximum sharpening everywhere. Combined with the additive G_e layer, this can slightly over-sharpen smooth transitions adjacent to texture, producing minor ringing. This is reflected in FSIM values slightly below Lanczos on pure-texture images.

Processing time on large images. The pixel-by-pixel edge-expansion loop (Equations 11–14) is implemented in Python and does not vectorise cleanly due to data-dependent writes. For a 2048×2048 upsampled image this becomes a bottleneck; a C extension or GPU implementation would be needed for real-time HD use.

CS convergence. With 10 nests and 20 generations, the Cuckoo Search occasionally converges to a local optimum, yielding a k that over-sharpens noisy regions. Increasing population size to 25 and generations to 50 improves PSNR on noisy images at the cost of roughly $3 \times$ longer execution.

6 Conclusion and Future Work

6.1 Conclusion

We presented OLA e-spline, an optimized edge-preserving image interpolation framework that chains Local Adaptive filtering, Unsharp Masking with Cuckoo Search gain optimisation, cubic B-spline interpolation, and additive edge expansion. Quantitative evaluation demonstrates consistent superiority over Bilinear, Bicubic, and Lanczos interpolation across PSNR, SSIM, and FSIM metrics at both $\times 2$ and $\times 4$ upscaling. The fixed- k variant (PSNR: 36.21 dB at $\times 2$, 31.63 dB at $\times 4$) offers a practical balance between quality and execution time, while the CS variant provides fully automatic gain tuning for applications where per-image optimality is paramount. The framework is simple, scale-agnostic, and implementable without training data, making it suitable for consumer electronics, broadcast upscaling, and medical imaging pipelines.

6.2 Future Work

- **Deep learning integration.** A convolutional network trained to predict the optimal G_e correction map could replace the hand-crafted Canny+directional-change pipeline, potentially recovering texture details beyond the reach of local operators.

- **Video super-resolution.** Temporal consistency constraints and motion-compensated edge expansion could extend OLA e-spline to video, where flickering artifacts are a key concern.
- **Real-time hardware implementation.** As analysed in the companion hardware feasibility study, the LA filter, USM, and B-spline stages map cleanly to FPGA line-buffer architectures. A fixed- k hardware pipeline could achieve 1080p upscaling at 30 fps on a mid-range FPGA.
- **Perceptual loss optimisation.** Replacing the Laplacian-variance CS fitness with a learned perceptual metric (LPIPS) may better align the optimised k with human visual judgement.
- **Medical and satellite imaging.** Evaluating on domain-specific datasets (CT/MRI slices, multispectral satellite imagery) would quantify OLA e-spline's value in the highest-stakes application domains.

6 References

- [1] J. Panda and S. Meher, "An improved image interpolation technique using OLA e-spline," *Egyptian Informatics Journal*, vol. 23, pp. 159–172, 2022.
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson Education, 2018.
- [3] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [4] X. Li and M. T. Orchard, "New edge-directed interpolation," *IEEE Transactions on Image Processing*, vol. 10, no. 10, pp. 1521–1527, 2001.
- [5] A. Giachetti and N. Asuni, "Real-time artifact-free image upscaling," *IEEE Transactions on Image Processing*, vol. 20, no. 10, pp. 2760–2768, 2011.
- [6] X. Liu *et al.*, "Improvement in directional cubic convolution image interpolation," in *SID Symposium Digest of Technical Papers*, vol. 51, pp. 455–458, 2020.
- [7] K. Zhang, D. Tao, X. Gao, X. Li, and Z. Xiong, "Learning multiple linear mappings for efficient single image super-resolution," *IEEE Transactions on Image Processing*, vol. 24, no. 3, pp. 846–861, 2015.
- [8] R. Nayak, B. K. Balabantaray, and D. Patra, "A new single-image super-resolution using efficient feature fusion and patch similarity in non-Euclidean space," *Arabian Journal for Science and Engineering*, vol. 45, no. 12, pp. 10261–10285, 2020.

-
- [9] Y. Xu, J. Li, H. Song, and L. Du, “Single-image super-resolution using panchromatic gradient prior and variational model,” *Mathematical Problems in Engineering*, 2021.
 - [10] X.-S. Yang and S. Deb, “Cuckoo search via Lévy flights,” in *Proc. World Congress on Nature & Biologically Inspired Computing*, pp. 210–214, 2009.
 - [11] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
 - [12] S. van der Walt *et al.*, “scikit-image: Image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014.